

Outlook of the unit

4 Ensemble methods

- Bagging
- Random forests
- Boosting

• •

Bagging

Bagging: Introduction

- ✓ The decision trees suffer from high variance.
- If we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different.
- A procedure with low variance will yield similar results if applied repeatedly to distinct data sets (like linear regression).
- Bootstrap aggregation, or bagging, is a general-purpose procedure for reducing the variance of a statistical learning method.
- ✓ Recall that given a set of n independent observations $Z_1, \dots, Z_n$ each with variance σ^2 , the variance of $\bar{Z}$ is given by σ^2/n .
- In other words, averaging a set of observations reduces variance.

- $X_1, X_2, \dots, X_n$ independent

$$E(X_i) = \mu \quad \text{Var}(X_i) = \sigma^2 < \infty$$

for all $i = 1, \dots, n$.

$$\text{Var}(X_1) = \sigma^2$$

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n}$$

$$n = 100$$

$$h = 10$$

$$\begin{aligned} \bar{X} &= \frac{1}{n} \sum X_i \\ &= \frac{X_1 + X_2 + \dots + X_n}{n} \end{aligned}$$

$$\underline{E(\bar{X}) = \mu}$$

Bootstrap.

DataID	Expenditure	Age	Gender
1	100	30	M
2	120	32	F
3	140	34	F
4	200	42	M
5	87	22	F
6	190	51	F

$n=6$: Sample Size

randomly
Selected
data id

A bootstrap sample

	Expenditure	Age	Gender
2	120	32	F
3	140	34	F
2	120	32	F
1	100	30	M
5	87	22	F
5	87	22	F

Bootstrap Sample 1

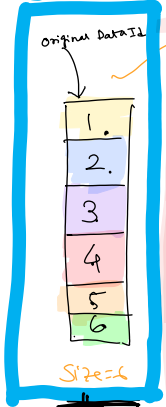
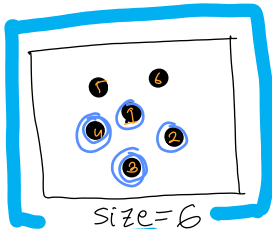
$n=6$

Bootstrap

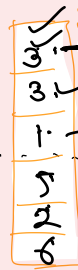
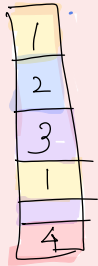
Sample 2.

$n=6$

We keep the Sample Size same as the original sample.



observed data



Sampling with Replacement
 Keeping Size same
 $n=6$

Bootstrap sample (B)

Bootstrap samples.

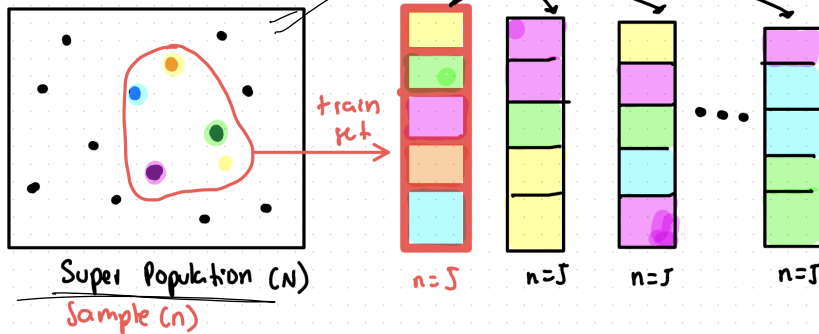
$b = 1, 2, \dots, B$

Bagging = Bootstrap and aggregation

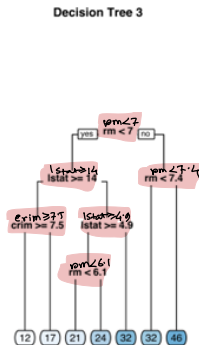
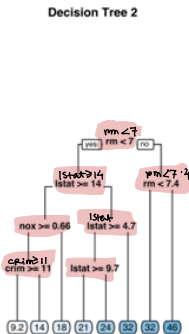
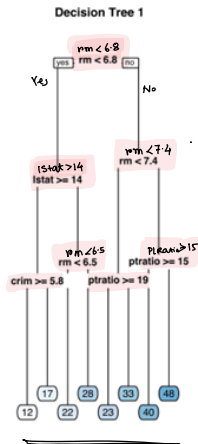
- A way to reduce the variance (increase prediction accuracy) of a method is to take many training sets, build prediction models using each training set, and average the predictions.
- This is not practical because we generally do not have access to multiple training sets.
- Instead, we can bootstrap, by taking repeated samples from the (single) training data set.

Bootstrapping idea

Bootstrapping: Random sampling with replacement



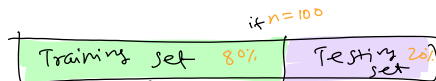
Using the well-known Boston housing data set, three decision trees are obtained based on three different sub-samples of the data.



Even if we alter a couple of rows of the data we may get entirely different tree based on different variables.

- The first few partitions are fairly similar at the top of each tree; however, they tend to differ substantially closer to the terminal nodes.
- These deeper nodes tend to overfit to specific attributes of the sample data; consequently, slightly different samples will result in highly variable estimate/predicted values in the terminal nodes.

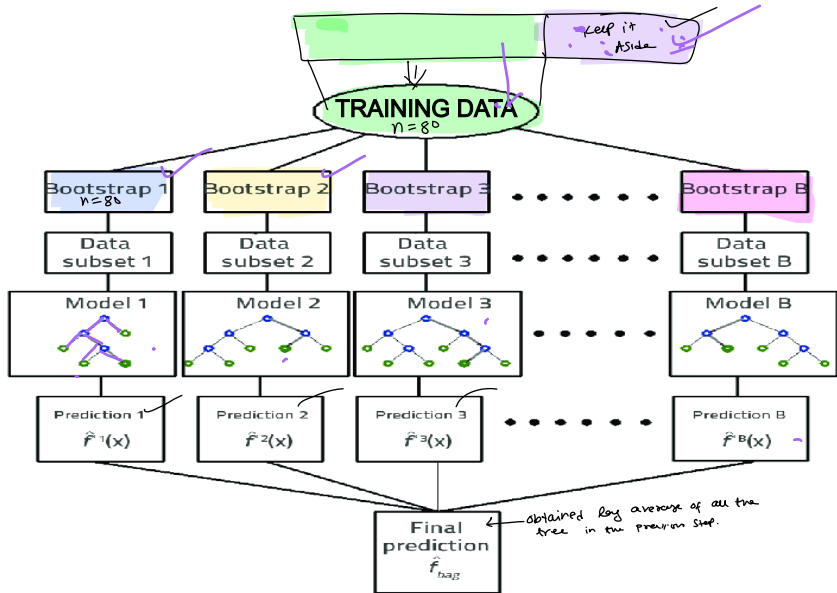
Bagging procedure for regression trees



- 1 Generate B different bootstrapped training data sets.
 $B = 5$. 5 different Bootstrap sample $n = 80$
- 2 Train the method on the b -th bootstrapped training set in order to get $\hat{f}^{*,b}(x)$ for $b = 1, \dots, B$ (for instance, construct regression trees without pruning).
- 3 Average all the predictions to obtain

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*,b}(x).$$

The Final output is based on averaging the output of all the trees generated from different Bootstrap samples



As an exercise, we may compare the accuracy of the output from any single tree on the Testing Set vs. the output of the bagged trees on Testing set.

Bagging in the context of classification trees

We have described the bagging procedure in the regression context, to predict a quantitative outcome Y .

Bagging for classification trees:

- 1 Generate B different bootstrapped training data sets.
- 2 Train the method on the b -th bootstrapped training set in order to record the class predicted by each of the B trees for a observation.
- 3 Take a majority vote: the overall prediction is the most commonly occurring majority class among the B predictions.

Comment:

Using a very large value of B will not lead to overfitting. In practice we use a value of B sufficiently large that the error has settled down, for instance $B = 500$.

Larger value for B is better.

However, more computation power is required if we choose a larger value for B .

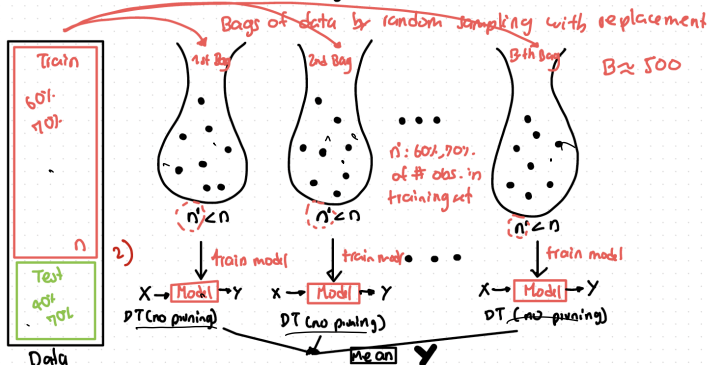
Variable importance measure

- When we bag a large number of trees, it is not possible to represent the resulting statistical learning procedure using a single tree.
- Thus, bagging improves prediction accuracy at the expense of interpretability.
- If there is a very strong predictor in a data set, then it is used to perform a first split in many trees grown from bagged samples.
- This leads to correlated bagged trees and averaging their predictions does not give substantial gains in accuracy.

Bagging idea

Bagging: Bootstrap aggregation (train each learner on a different set)

1) Generate B different training data sets



● Which features are important in making our prediction?

● What is the relationship between a feature and the target?

As the final output is based on averaging of multiple trees, the importance of a single variable in different trees are different. Thus, we need to incorporate the scores/reduction in MSE/Error from all the trees.

~~Which~~ Which features are important in making our prediction?

For a single tree, we can add up the decreases in the RSS associated with all splits involving a particular variable. For random forests/bagged trees, we simply aggregate these over all trees in the forest.

The variable that reduces the maximum amount of MSE is most important.

There are functions importance and varImpPlot in the randomForest package to do this.

Residual
sum of squares
MSE

What is the relationship between a feature and the target?

One way is to consider the average value of the prediction function for a fixed value of a chosen variable, averaged over the other variables.

We can repeat this for different values of x_1 and plot the result. This is called a Partial Dependence Plot.

Bagging regression trees in R: Boston housing data

The `randomForest()`-command constructs a bagged tree and returns the results as an object.

```
# Loading libraries and the data
```

```
library(randomForest)
```

```
# Bagging a regression tree with all variables
```

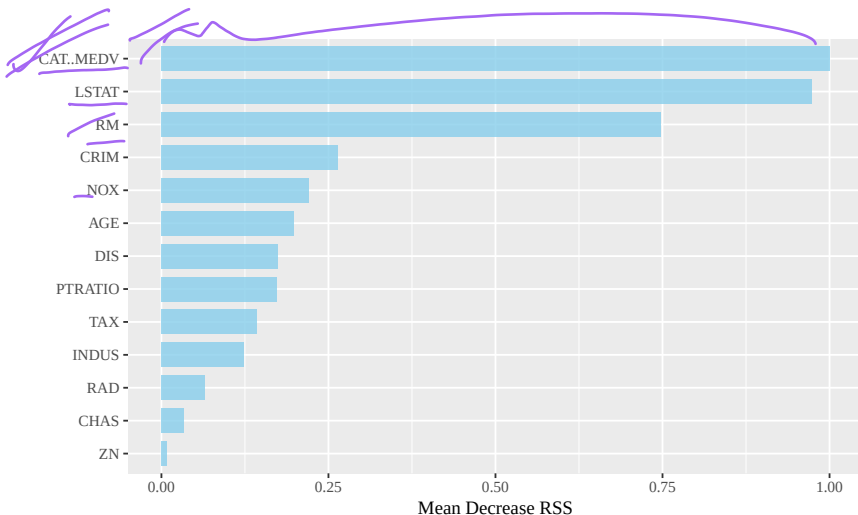
```
> fit <- randomForest(MEDV ~ ., data = BostonH, mtry = 13,  
  importance = TRUE, subset = inTrain)  
> fit
```

Call:

```
randomForest(formula = MEDV ~ ., data = BostonH, mtry =  
  13, importance = TRUE, subset = inTrain)  
Type of random forest: regression  
Number of trees: 500  
No. of variables tried at each split: 13
```

- `mtry` indicates that all 13 predictors should be considered for each split of the tree.

Bagging regression trees in R: Variable importance measure

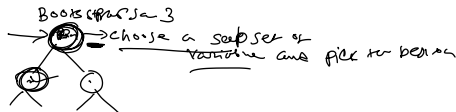


Random forests

Bagging to random forests

- If there is a very strong predictor in a data set, then it is used to perform a first split in many trees grown from bagged samples.
- Consequently, all of the bagged trees will look quite similar to each other.
- This leads to correlated bagged trees and averaging their predictions does not give substantial gains in accuracy.
- Random forests provide an improvement over bagged trees
- It offers a solution to correlated bagged trees: allows a random sample of m out of p predictors at each split in a tree in order to expose different features to be explored by an algorithm.

Random forests



- We build a number of decision trees on bootstrapped training samples.
- When building these decision trees, each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors.
- The split is allowed to use only one of those m predictors.
- A fresh selection of m predictors is taken at each split, and typically we choose $m \approx \sqrt{p}$ or by cross-validation.
- If a random forest is built using $m = p$, then this amounts simply to bagging.

Random forests in R: Boston housing data

The `randomForest()`-command constructs random forests and returns the results as an object.

```
# Bagging a regression tree with all variables
> fit<-randomForest(MEDV_Fac~.,data=BostonH,mtry=sqrt(12)
  , importance =TRUE,subset=inTrain)
> fit
```

Type of random forest: classification

Number of trees: 500

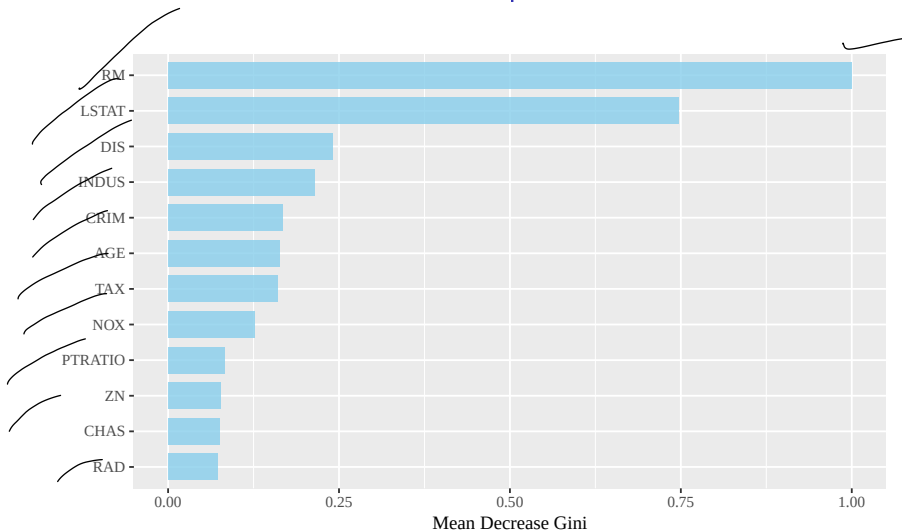
No. of variables tried at each **split**: 3

OOB estimate of error rate: 3.93%

Confusion **matrix**:

	Below	Above	class.error
Below	289	5	0.0170068
Above	9	53	0.1451613

Random forests in R: Variable importance measure



Pros and cons of random forest

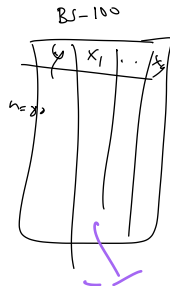
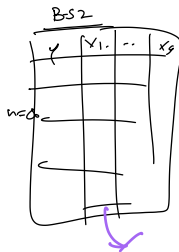
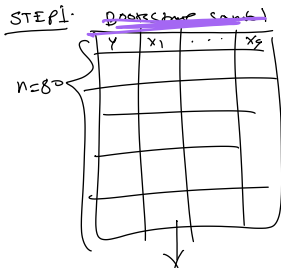
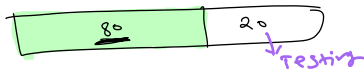
Pros

- high predictive performance
- low chance of overfitting
- robustness
- easy to parallelize

Cons

- relatively long training time
- lack of interpretability
(possible solution: variable importance measures)
- cannot incorporate missing values as easily as simple trees

$Y, X_1, X_2, X_3, X_4, X_5, X_6, X_7, X_8, X_9$



• STEP 2.



choose a random set of $3 = \sqrt{9}$ variables.

X_5, X_7, X_8

• pick best variable among $\{X_5, X_7, X_8\}$

using the criteria that we use in regression trees



We select a random 3 variable X_3, X_1, X_5



random chose 3 var out of 9
 X_3, X_1, X_2

Boosting

Boosting

Boosting is another technique for improving predictive accuracy of decision trees. It 'boosts' the performance of weak (only marginally better than random) classifiers.

Unlike bagging, in boosting trees are grown sequentially: a tree uses information from previously grown trees. Final classifier is built by combining sequentially updated classifiers.

Some popular boosting approaches:

- AdaBoost
- Stochastic Gradient Boosting

AdaBoost

Process:

- 1 Initially each observation has the same starting weight ($1/n$), an initial 'base' classifier is built and misclassification error is calculated.
- 2 Weights are updated in such a way that incorrectly predicted observations receive more weight and correctly classified less. A new classifier with updated weights is built and this step is repeated M times.
- 3 Final classifier is calculated as a weighted sum of M base classifiers.

Tuning parameters for boosting

- **Number of trees** should not be too large, because unlike bagging and random forest, boosting can overfit.
- **Shrinkage parameter, or learning rate** controls speed of learning. Typical values are 0.1 or 0.01.
- **Interaction depth** is the number of splits in each tree. One split per tree often works well.
- **Subsampling** uses only part of the data (typically $\frac{1}{2}$ or less) to improve predictive performance.

Use cross-validation to find promising values.

AdaBoost (Freund & Schapire, 1996)

For this slide assume we have binary y . $y \in \{+1, -1\}$. $f(x_i) \in \{+1, -1\}$.

1. Initialize the observation weights $w_i = \frac{1}{N}$, $i = 1, 2, \dots, N$.
2. For $m = 1$ to M repeat steps (a)–(d):
 - (a) Fit a classifier $f_m(x)$ to the training data using weights w_i .
 - (b) Compute weighted error of newest tree

$$\mathcal{E}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq f_m(x_i))}{\sum_{i=1}^N w_i}.$$

- (c) Compute $\alpha_m = \log\left[\frac{1-\mathcal{E}_m}{\mathcal{E}_m}\right] = \logit(1-\mathcal{E}_m) = \logit(\text{Accuracy}_m)$
 - (d) Update weights for $i = 1, \dots, N$:
$$w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq f_m(x_i))]$$
and renormalize to w_i to sum to 1.
3. Output $\hat{f}(x) = \text{sign}\left[\sum_{m=1}^M \alpha_m f_m(x)\right]$.

1

2

m=1

m=2

m=3

...

m=M

3

i=1

i=2

i=3

...

i=N

	(x_1, y_1) $w_1^{(1)} = 1$	(x_2, y_2) $w_2^{(1)} = 1$	(x_3, y_3) $w_3^{(1)} = 1$	(x_i, y_i) $w_i^{(1)} = 1$	(x_N, y_N) $w_N^{(1)} = 1$	
	$w_1^{(1)} = 1$	$w_2^{(1)} = 1$	$w_3^{(1)} = 1$	$w_i^{(1)} = 1$	$w_N^{(1)} = 1$	$\Rightarrow \epsilon_1 = \frac{\sum_{i=1}^N w_i^{(1)} I(y_i \neq f_1(x_i))}{\sum_{i=1}^N w_i^{(1)}}$ $\Leftarrow \alpha_1 = \log\left(\frac{1 - \epsilon_1}{\epsilon_1}\right)$
m=1	$w_1^{(2)} = w_1^{(1)}$ $w_1^{(2)} = w_1^{(1)} \text{ odds}(1 - \epsilon_1) \text{ if } y_1 \neq f_1(x_1)$ $w_1^{(2)} = w_1^{(1)} \text{ if } y_1 = f_1(x_1)$	$w_2^{(2)} = w_2^{(1)} \exp(\alpha_1 I(y_2 \neq f_1(x_2)))$ $w_2^{(2)} = \begin{cases} w_2^{(1)} \text{ odds}(1 - \epsilon_1) & \text{if } y_2 \neq f_1(x_2) \\ w_2^{(1)} & \text{if } y_2 = f_1(x_2) \end{cases}$	$w_3^{(2)} = w_3^{(1)}$ $w_3^{(2)} = \begin{cases} w_3^{(1)} \text{ odds}(1 - \epsilon_1) & \text{if } y_3 \neq f_1(x_3) \\ w_3^{(1)} & \end{cases}$	$w_i^{(2)} = w_i^{(1)}$	$w_N^{(2)} = w_N^{(1)}$	
m=2	$w_1^{(3)} = w_1^{(2)}$ $w_1^{(3)} = w_1^{(2)} \text{ odds}(1 - \epsilon_2) \text{ if } y_1 \neq f_2(x_1)$ $w_1^{(3)} = w_1^{(2)} \text{ if } y_1 = f_2(x_1)$	$w_2^{(3)} = w_2^{(2)} \text{ odds}(1 - \epsilon_2) \text{ if } y_2 \neq f_2(x_2)$ $w_2^{(3)} = w_2^{(2)} \text{ if } y_2 = f_2(x_2)$	$w_3^{(3)} = w_3^{(2)}$	$w_i^{(3)} = w_i^{(2)}$	$w_N^{(3)} = w_N^{(2)}$	$\Rightarrow \epsilon_2 = \frac{\sum_{i=1}^N w_i^{(2)} I(y_i \neq f_2(x_i))}{\sum_{i=1}^N w_i^{(2)}}$ $\Leftarrow \alpha_2 = \log\left(\frac{1 - \epsilon_2}{\epsilon_2}\right) = \log\left(\frac{1 - \epsilon_2}{1 - \epsilon_2}\right)$
m=3	$w_1^{(4)} = w_1^{(3)}$ $w_1^{(4)} = w_1^{(3)} \text{ odds}(1 - \epsilon_3) \text{ if } y_1 \neq f_3(x_1)$ $w_1^{(4)} = w_1^{(3)} \text{ if } y_1 = f_3(x_1)$	$w_2^{(4)} = w_2^{(3)} \text{ odds}(1 - \epsilon_3) \text{ if } y_2 \neq f_3(x_2)$ $w_2^{(4)} = w_2^{(3)} \text{ if } y_2 = f_3(x_2)$	$w_3^{(4)} = w_3^{(3)}$	$w_i^{(4)} = w_i^{(3)}$	$w_N^{(4)} = w_N^{(3)}$	$\epsilon_3 =$ $\alpha_3 = \log\left(\frac{1 - \epsilon_3}{\epsilon_3}\right)$
...						
m=M						$\epsilon_M =$ $\alpha_M = \log\left(\frac{1 - \epsilon_M}{\epsilon_M}\right)$

$$\hat{f}(x) = \text{Sign}\left(\sum_{m=1}^M \alpha_m f_m(x)\right)$$

Pros and cons of boosted trees

Pros

- very high predictive performance if tuning parameters are chosen wisely
- each iteration improves the solution
- easily incorporates missing values
- can deal with larger data sets than random forest

Cons

- harder to tune because of larger number of parameters
- considerable chance of overfitting
- relatively long training time
- lack of interpretability (possible solution: variable importance measures)

Summary

- Decision trees are simple and interpretable models for regression and classification.
- However they are often not competitive with other methods in terms of prediction accuracy.
- Bagging, random forests (and boosting) are good methods for improving the prediction accuracy of trees.
- They work by growing many trees on the training data and then combining the predictions of the resulting ensemble of trees.
- Random forests and boosting are among the state-of-the-art methods for supervised learning - however their results can be difficult to interpret.